Synaptic Plasticity as Short-Term Memory, and Where It Lives in BDH

One-page concept summary — DataForge 2026, Pathway Track

Modern language models keep track of what they have already seen in one of two ways: a Transformer stores a growing key–value cache of every past token and compares the current token against all of it (attention), while recurrent and state-space alternatives compress the past into a fixed-size state that is read and rewritten as new tokens arrive. Both approaches treat memory as a data structure sitting outside the network's learned parameters. A much older idea from computational neuroscience offers a third option: let the network's own connection strengths change during use. In a Hebbian system, two units that are active together have the synapse between them strengthened by a simple local rule, often written as an outer product of their activities, $w_{ij} \leftarrow \lambda \cdot w_{ij} + x_i \cdot x_j$. No gradient step, no backward pass, and no external buffer is involved — the network's wiring becomes a working memory that holds several recently seen patterns at once, and a later partial or noisy cue can be used to reconstruct one of them by letting the network settle toward a stored state (classical associative, or 'Hopfield-style', memory).

What changes, and what it trades away

The mechanism changes where adaptation happens: instead of a separate memory module bolted onto a static network, storage and computation share the same substrate. This buys fast, gradient-free, per-context memory that requires no additional parameters. It trades away unbounded capacity: for a network of N units, the number of patterns that can be stored and later recalled without significant errors is limited (classical dense Hopfield capacity scales roughly linearly with N, with a small constant of proportionality). Beyond that point, stored patterns interfere — retrieval can settle on a blend of memories or a spurious state rather than degrading gracefully. A key finding across the associative-memory and fast-weight literature is that constraining the stored activity to be sparse and non-negative substantially reduces this interference for a given network size, because fewer simultaneously active units share fewer connections across different stored patterns.

Representative systems

Classical associative memory / Hopfield networks established the outer-product storage-and-settling rule this concept is built on, but store dense, often bipolar patterns and have well-known, comparatively low capacity relative to network size. **Linear-attention / fast-weight programmers** (Schlag, Irie & Schmidhuber, 2021; extended by Irie et al., ICML 2022) showed that linearized self-attention is formally an outer-product fast-weight write followed by a query-based read — the same Hebbian mathematics, reframed as a sequence-model mechanism, with follow-up work adding a delta rule to actively correct for capacity overload rather than simply summing writes. **Dragon Hatchling (BDH)** (Kosowski et al., Pathway, Sept. 2025, arXiv:2509.26507) is a recent, large-scale instantiation of this family: it replaces the Transformer's key–value cache with a scale-free graph of neuron-like units whose synapses are updated via a Hebbian rule as the model reads, and it is reported to match GPT-2-scale Transformer performance from 10M to 1B parameters on language and translation tasks while remaining a practical, GPU-friendly architecture (BDH-GPU).

Where BDH and BDH-CQ demonstrate the concept

BDH's working memory is reported to be the state of its synapse graph itself, with two properties directly relevant to the interference trade-off above: activation vectors are sparse and positive, with roughly 5% of neurons active and activity level varying with input predictability rather than a fixed budget; and individual synapses are reported to become monosemantic — selective for a single recognisable concept — which is the graph-level reason sparse Hebbian writes interfere less. **BDH-CQ** (Pathway, Aug. 2026, arXiv:2608.09888) is a 150M-parameter reasoning variant that applies the same family of mechanism to in-context learning: task demonstrations update a recurrent contextual memory additively, the paper explicitly relates this to fast-weight and linear-attention views of contextual association, and no parameters are changed at inference — the model then solves a held-out query through iterative computation in a continuous latent workspace, without a written chain of thought. On the public ARC-AGI-1 evaluation set, BDH-CQ is reported to reach 29.5% pass@2 (24.25% pass@1) at an estimated cost of about $0.0007 per task, a result the authors frame as a cost–accuracy trade-off rather than a claim of state-of-the-art accuracy. These are the developer's own reported benchmark results, not an independent third-party reproduction or a production deployment, and should be read as such.

Evidence and limitations

The Hebbian-write, capacity-limited, sparsity-helps-interference pattern is well established across the classical associative-memory and fast-weight-programmer literature, and is independently corroborated by at least two 2025–2026 follow-up papers applying Hebbian plasticity to Transformer memory modules. BDH's specific numbers (sparsity level, monosemanticity, scaling behaviour) come from a single research group's paper and have not yet, to the team's knowledge, been independently reproduced at scale; BDH-CQ's ARC-AGI results are likewise self-reported. The most important open question is how BDH's within-context synaptic state relates to durable, cross-session learning: the paper's within-context memory should not be read as evidence that the architecture solves catastrophic forgetting or long-term continual learning more broadly — that is a distinct, still-open problem the primary papers themselves do not claim to resolve.

Sources: Kosowski et al., arXiv:2509.26507 (2025) · Pathway, arXiv:2608.09888 (2026) · Schlag, Irie & Schmidhuber, arXiv:2102.11174 (2021) · Irie et al., ICML 2022 · Chaudhary, arXiv:2510.21908 (2025) · arXiv:2605.02920 (2026). See the accompanying interactive artifact and README for the full reference list and an interactive, live demonstration of the Hebbian storage/recall/interference mechanism described above.